

# Assignment 2: CircuitVerse Gates and Adders

**Name:** Hiram Mora Saucedo

**RedID:** 132765669

**Course / Section:** CS 240

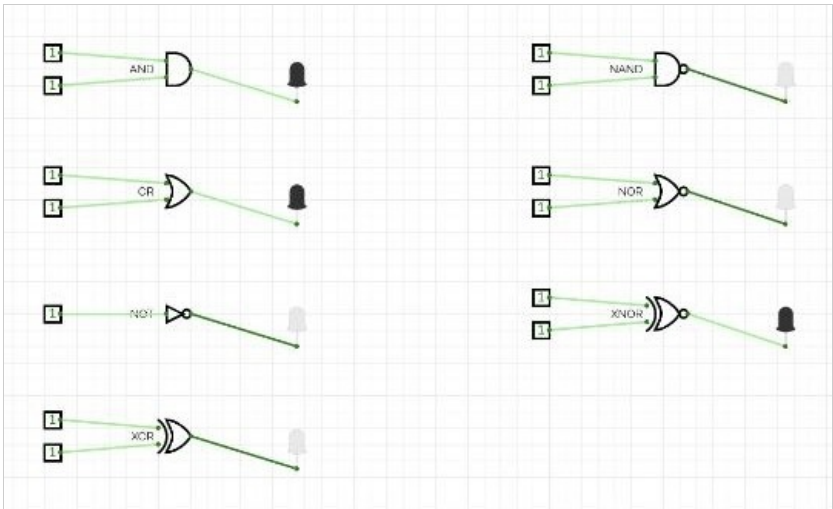
**CircuitVerse link:** <https://circuitverse.org/users/460756/projects/assignment-2-gates-and-adders-46016ef2-7d7e-47b2-aa63-e3b3aa9aabf3>

**GitHub repository:** <https://github.com/hmorasaucedo2166-boop/circuitverse-assignment2-gates-adders>

## Part A - Gates

### Basic gates

| A | B | AND | OR | XOR | XNOR | NAND | NOR | NOT A |
|---|---|-----|----|-----|------|------|-----|-------|
| 0 | 0 | 0   | 0  | 0   | 1    | 1    | 1   | 1     |
| 0 | 1 | 0   | 1  | 1   | 0    | 1    | 0   | 1     |
| 1 | 0 | 0   | 1  | 1   | 0    | 1    | 0   | 0     |
| 1 | 1 | 1   | 1  | 0   | 1    | 0    | 0   | 0     |



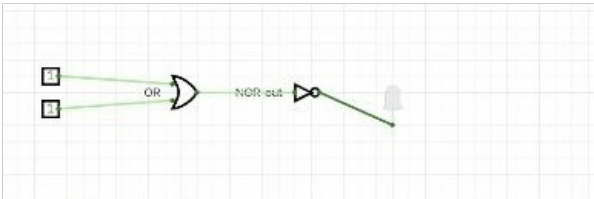
All 7 gates wired to switches/LEDs.

### Custom NAND, NOR, XNOR

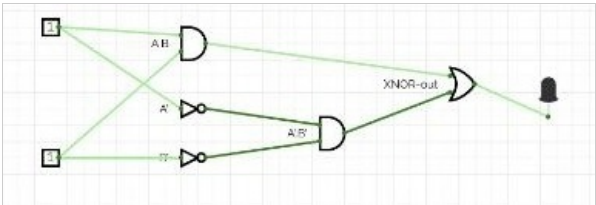
$\text{NAND} = \text{NOT}(\text{A} \cdot \text{B})$ .  $\text{NOR} = \text{NOT}(\text{A} + \text{B})$ .  $\text{XNOR} = (\text{A} \cdot \text{B}) + (\text{A}' \cdot \text{B}')$ .



Custom NAND (AND + NOT), wired to an LED.



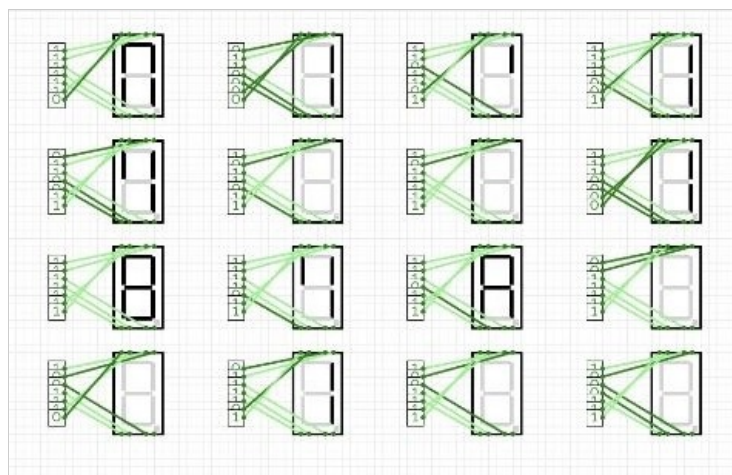
Custom NOR (OR + NOT), wired to an LED.



Custom XNOR (AND/OR/NOT combo), wired to an LED.

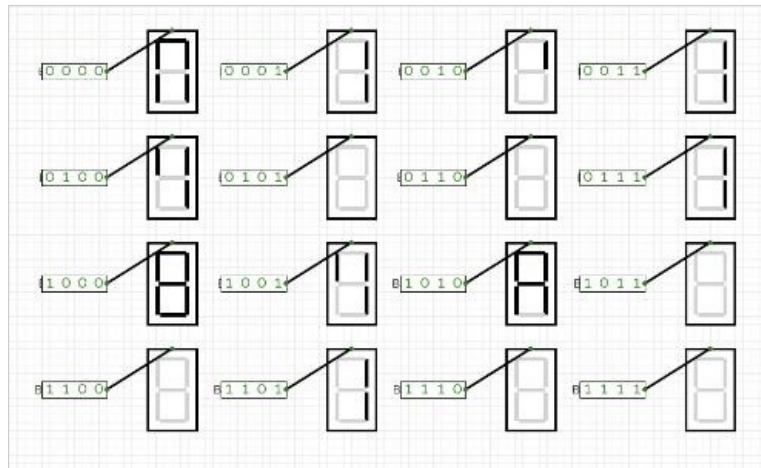
## Part B - Displays

### Manual hex display



All 16 values, 0-F.

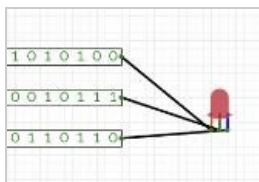
## Smart hex display



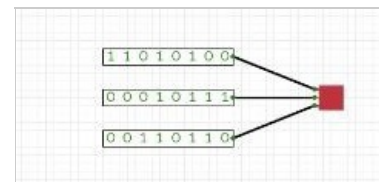
All 16 values, 0-F.

## RGB LED - SDSU Red

SDSU Red #D41736 = R 212, G 23, B 54.

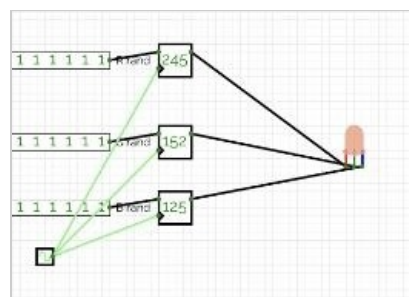


Round RGB LED.



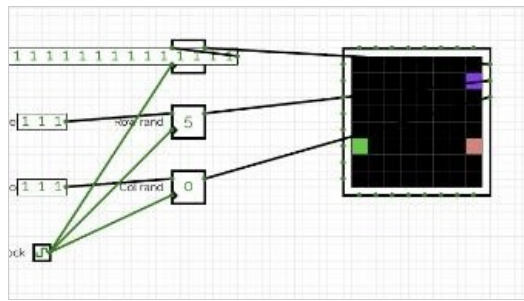
Square RGB LED.

## Random colored LED



Randomizers + clock into RGB LED.

## Random RGB matrix



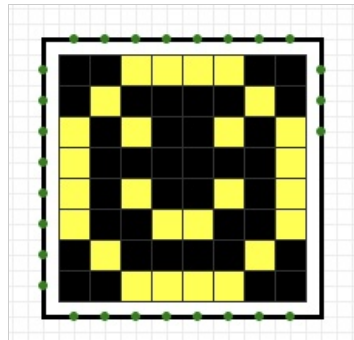
Randomizers + clock into matrix.

## Smiley face

```

. . Y Y Y Y . .
. Y . . . . Y .
Y . Y . . Y . Y
Y . . . . . Y
Y . Y . . Y . Y
Y . . Y Y . . Y
. Y . . . . Y .
. . Y Y Y Y . .

```



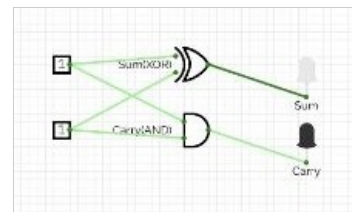
Smiley face on RGB matrix.

## Part C - Advanced Logic

### Half adder and full adder

Half adder:  $\text{Sum} = A \text{ XOR } B$ ,  $\text{Carry} = A \text{ AND } B$ .

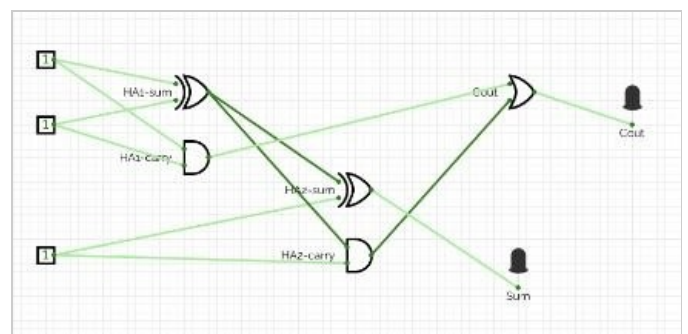
| A | B | Sum | Carry |
|---|---|-----|-------|
| 0 | 0 | 0   | 0     |
| 0 | 1 | 1   | 0     |
| 1 | 0 | 1   | 0     |
| 1 | 1 | 0   | 1     |



Half adder.

Full adder:  $\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$ ,  $\text{Cout} = (A \cdot B) + (\text{Cin} \cdot (A \text{ XOR } B))$ .

| A | B | Cin | Sum | Cout |
|---|---|-----|-----|------|
| 0 | 0 | 0   | 0   | 0    |
| 0 | 0 | 1   | 1   | 0    |
| 0 | 1 | 0   | 1   | 0    |
| 0 | 1 | 1   | 0   | 1    |
| 1 | 0 | 0   | 1   | 0    |
| 1 | 0 | 1   | 0   | 1    |
| 1 | 1 | 0   | 0   | 1    |
| 1 | 1 | 1   | 1   | 1    |

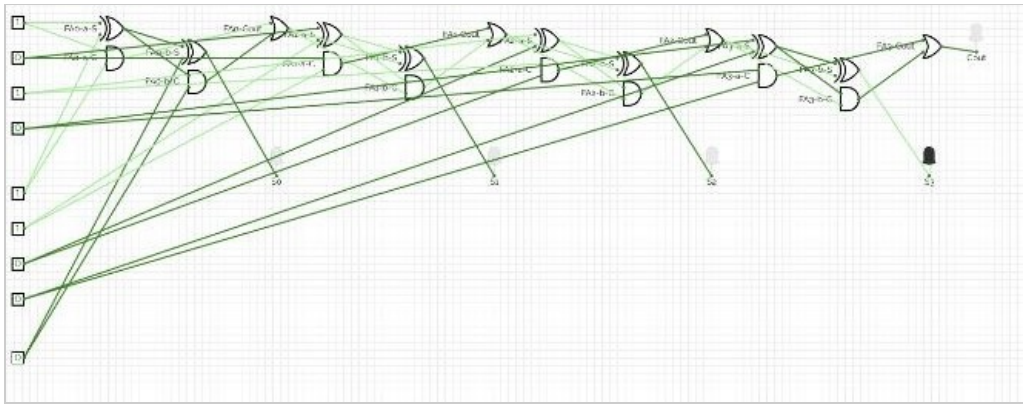


Full adder.

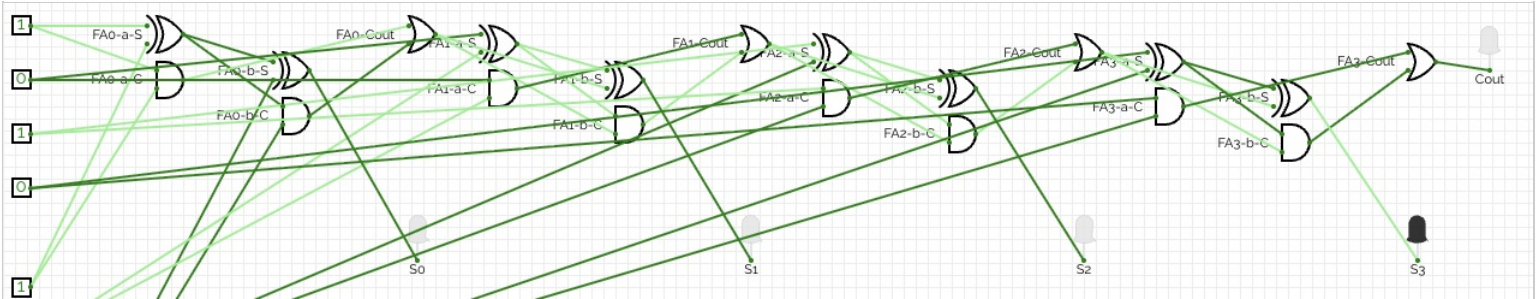
### 4-bit ripple-carry adder

Four full adders chained, carry out feeds next carry in.

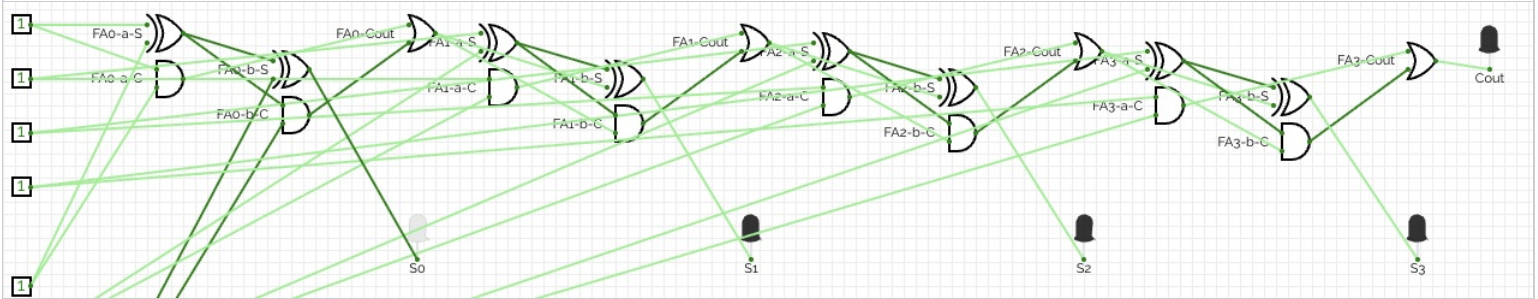
| A + B   | Result | S3S2S1S0 | Cout |
|---------|--------|----------|------|
| 0 + 0   | 0      | 0000     | 0    |
| 5 + 3   | 8      | 1000     | 0    |
| 9 + 6   | 15     | 1111     | 0    |
| 7 + 8   | 15     | 1111     | 0    |
| 15 + 1  | 16     | 0000     | 1    |
| 15 + 15 | 30     | 1110     | 1    |



4-bit ripple-carry adder.



Test: 5 + 3 = 8

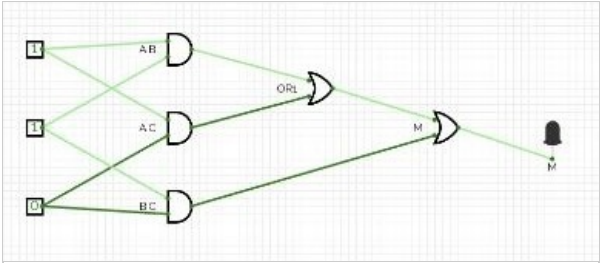


Test: 15 + 15 = 30

Original circuit: majority voter

M = (A·B) + (A·C) + (B·C). Output is 1 when at least 2 of 3 inputs are 1.

| A | B | C | M |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |



Majority voter.

Design explanation

All of the circuits are combinational, which means that their outputs depend only on the current inputs and do not use memory or feedback. I tested each standard and custom gate using its truth table. I also verified the half adder and full adder with every

possible input combination before connecting them to build the 4 bit ripple carry adder. I tested the 4 bit adder with six cases, including 0 + 0 as a baseline test, 15 + 1 and 15 + 15 to check carry propagation and overflow. I finally I tested the majority voter using all eight possible input combinations.

## Source code

The full project source is the raw CircuitVerse project file (`Assignment2_Gates_and_Adders.cv`), included in the GitHub repository linked on the cover page. Because a CircuitVerse project is a JSON description of wire coordinates rather than written code, the full raw file is not reproduced here (it is ~12,700 lines of coordinate data); instead, below is a readable listing of every circuit and its logic elements, generated directly from that file.

```
Circuit: A1_Basic_Gates
Elements: 1x AndGate, 7x DigitalLed, 13x Input, 1x NandGate, 1x NorGate, 1x NotGate, 1x OrGate, 1x XnorGate, 1x XorGate
Named / logic elements:
- NotGate "NOT" (1-input)
- OrGate "OR" (2-input)
- AndGate "AND" (2-input)
- NorGate "NOR" (2-input)
- NandGate "NAND" (2-input)
- XorGate "XOR" (2-input)
- XnorGate "XNOR" (2-input)
```

```
Circuit: A2_Custom_Gates
Elements: 3x AndGate, 3x DigitalLed, 6x Input, 4x NotGate, 2x OrGate
Named / logic elements:
- NotGate "NAND-out" (1-input)
- NotGate "NOR-out" (1-input)
- NotGate "A'" (1-input)
- NotGate "B'" (1-input)
- OrGate "OR" (2-input)
- OrGate "XNOR-out" (2-input)
- AndGate "AND" (2-input)
- AndGate "A.B" (2-input)
- AndGate "A'.B'" (2-input)
```

```
Circuit: B1_Manual_Hex
Elements: 112x ConstantVal, 16x SevenSegDisplay
```

```
Circuit: B2_Smart_Hex
Elements: 16x ConstantVal, 16x HexDisplay
Named / logic elements:
- ConstantVal "B=0" (value = 0000 (binary, 4-bit))
- ConstantVal "B=1" (value = 0001 (binary, 4-bit))
- ConstantVal "B=2" (value = 0010 (binary, 4-bit))
- ConstantVal "B=3" (value = 0011 (binary, 4-bit))
- ConstantVal "B=4" (value = 0100 (binary, 4-bit))
- ConstantVal "B=5" (value = 0101 (binary, 4-bit))
- ConstantVal "B=6" (value = 0110 (binary, 4-bit))
- ConstantVal "B=7" (value = 0111 (binary, 4-bit))
- ConstantVal "B=8" (value = 1000 (binary, 4-bit))
- ConstantVal "B=9" (value = 1001 (binary, 4-bit))
- ConstantVal "B=10" (value = 1010 (binary, 4-bit))
- ConstantVal "B=11" (value = 1011 (binary, 4-bit))
- ConstantVal "B=12" (value = 1100 (binary, 4-bit))
- ConstantVal "B=13" (value = 1101 (binary, 4-bit))
- ConstantVal "B=14" (value = 1110 (binary, 4-bit))
- ConstantVal "B=15" (value = 1111 (binary, 4-bit))
```

```
Circuit: B3_B4_RGB_LEDs
Elements: 6x ConstantVal, 1x RGBLed, 1x SquareRGBLed
Named / logic elements:
- ConstantVal "R=212" (value = 11010100 (binary, 8-bit))
- ConstantVal "G=23" (value = 00010111 (binary, 8-bit))
- ConstantVal "B=54" (value = 00110110 (binary, 8-bit))
- ConstantVal "R=212" (value = 11010100 (binary, 8-bit))
- ConstantVal "G=23" (value = 00010111 (binary, 8-bit))
- ConstantVal "B=54" (value = 00110110 (binary, 8-bit))
```

```
Circuit: B5_Random_LED
Elements: 1x Clock, 3x ConstantVal, 1x RGBLed, 3x Random
Named / logic elements:
- ConstantVal "max=255" (value = 11111111 (binary, 8-bit))
- ConstantVal "max=255" (value = 11111111 (binary, 8-bit))
- ConstantVal "max=255" (value = 11111111 (binary, 8-bit))
- Clock "Clock"
- Random "R rand"
- Random "G rand"
- Random "B rand"
```

```
Circuit: B6_Random_Matrix
Elements: 1x Clock, 3x ConstantVal, 1x RGBLedMatrix, 3x Random
Named / logic elements:
- ConstantVal "max color" (value = 11111111111111111111111111111111 (binary, 24-bit))
- ConstantVal "max row" (value = 111 (binary, 3-bit))
- ConstantVal "max col" (value = 111 (binary, 3-bit))
- Clock "Clock"
- Random "Color rand"
- Random "Row rand"
- Random "Col rand"
```

```
Circuit: B7_Smiley_Face
Elements: 1x RGBLedMatrix
```

```
Circuit: C1_Half_Full_Addder
Elements: 3x AndGate, 4x DigitalLed, 5x Input, 1x OrGate, 3x XorGate
Named / logic elements:
- Input "A" (initial value = 1)
- Input "B" (initial value = 1)
- Input "A" (initial value = 1)
- Input "B" (initial value = 1)
- Input "Cin" (initial value = 1)
- OrGate "OR (Carry out)" (2-input)
- AndGate "AND (Carry)" (2-input)
- AndGate "AND1" (2-input)
- AndGate "AND2" (2-input)
- XorGate "XOR (Sum)" (2-input)
- XorGate "XOR1" (2-input)
- XorGate "XOR2 (Sum)" (2-input)
- DigitalLed "Sum"
- DigitalLed "Carry"
- DigitalLed "Sum"
- DigitalLed "Cout"
```

```
Circuit: C2_Ripple_Carry_Addder
Elements: 8x AndGate, 5x DigitalLed, 9x Input, 4x OrGate, 8x XorGate
Named / logic elements:
- Input "A0" (initial value = 1)
- Input "B0" (initial value = 0)
- Input "Cin" (initial value = 0)
- Input "A1" (initial value = 0)
- Input "B1" (initial value = 1)
- Input "A2" (initial value = 0)
- Input "B2" (initial value = 1)
- Input "A3" (initial value = 1)
- Input "B3" (initial value = 0)
- OrGate (2-input)
- OrGate (2-input)
- OrGate (2-input)
- OrGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- AndGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- XorGate (2-input)
- DigitalLed "S0"
- DigitalLed "S1"
- DigitalLed "S2"
- DigitalLed "S3"
- DigitalLed "Cout"
```

```
Circuit: C3_Majority_Voter
Elements: 3x AndGate, 1x DigitalLed, 3x Input, 1x OrGate
Named / logic elements:
- Input "A" (initial value = 1)
- Input "B" (initial value = 1)
- Input "C" (initial value = 0)
- OrGate "Majority" (3-input)
- AndGate "A.B" (2-input)
- AndGate "B.C" (2-input)
- AndGate "A.C" (2-input)
- DigitalLed "Output"
```

**Sources**

AI use: I watched multiple youtube videos/tutorials and used chatgpt to help learn and understand the circuitverse tools and the gate/adder concepts before building the circuits myself. After building and testing everything, I gave my screenshots and truth table data to Claude to help organize them into this formatted PDF document.